

BÀI TẬP VỀ NHÀ — Ôn tập Bài 7 → 12

Phạm vi ôn tập: Giai đoạn 3 — Kiến trúc Component

Nguyên tắc quan trọng: KHÔNG động vào project Smart Recipe Box. Toàn bộ bài tập làm trên project riêng `bai-tap-tra-sua`, để dự án chính giữ nguyên trạng thái sẵn sàng cho Giai đoạn 4.

Chuẩn bị

Nếu bạn CHƯA có project `bai-tap-tra-sua`

```
cd <thư mục chứa project của bạn>
```

```
ng new bai-tap-tra-sua --style=css --ssr=false --routing=false --skip-tests
```

```
cd bai-tap-tra-sua && ng serve --port 4300
```

 Chọn `--style=css` (CSS thuần) chứ **ĐỪNG chọn Tailwind**, vì **Nhiệm vụ C10** là bài tập tự viết CSS bằng tay. Nếu bạn đã tạo project với Tailwind rồi thì vẫn làm được, chỉ cần khi tới C10 thì viết CSS thuần vào file `.css` như bình thường.

Nếu bạn ĐÃ làm bài tập 1–6

Bỏ qua phần code khởi đầu bên dưới, dùng luôn project cũ của bạn.

Code khởi đầu (trạng thái tương đương "đã xong Bài 6")

Tạo 2 file trong `src/app/` và dán nội dung vào. Sau đó thay nội dung `app.ts` và `app.html`.

-  `src/app/models.ts` (bấm để mở)
-  `src/app/mock-drinks.ts` (bấm để mở)
-  `src/app/app.ts` (bấm để mở)
-  `src/app/app.html` (bấm để mở)

Chạy `ng serve --port 4300`, thấy trang hiện ra là bạn đã sẵn sàng bắt đầu.

PHẦN A — Lý thuyết

Trả lời ngắn gọn bằng lời của bạn.

1. Phân biệt `{{ }}` và `[ ]`. Cả hai đều đưa dữ liệu ra màn hình, vậy tại sao cần tới hai cái?
2. Vì sao `<button disabled="{{ isLocked() }}">` là sai, còn `<button [disabled]="isLocked()">` là đúng?
3. Trong ví dụ `<img [src]="drink().imageUrl" width="300">`, tại sao `src` có ngoặc vuông mà `width` thì không?
4. Kể ra **ba bước** để dùng một component con bên trong component cha. Thiếu bước nào thì chuyện gì xảy ra?
5. Vì sao component `App` nằm thẳng trong `src/app/`, còn các component khác lại có thư mục riêng?
6. Khi code chuyển từ `src/app/app.ts` xuống `src/app/drink-list/drink-list.ts`, tại sao `import { DrinkModel } from './models'` phải đổi thành `import { DrinkModel } from '../models'`?
7. Vì sao một `input()` **không** được đặt `protected`, trong khi signal thường thì nên có?
8. Phân biệt `[drink]="selectedDrink()"` và `[drink]="selectedDrink"`. Cái nào đúng và tại sao?
9. CSS viết trong `drink-detail.css` có ảnh hưởng tới thẻ `<h2>` ở component khác không? Giải thích cơ chế.
10. `@if` và CSS `display: none` khác nhau ở điểm nào? Nêu một tình huống mà sự khác biệt đó thật sự quan trọng.
11. `track` trong `@for` dùng để làm gì? Vì sao `track $index` thường là lựa chọn tệ?
12. Có cần import gì để dùng `@for` và `@if` không? So sánh với `*ngFor` ngày xưa.

PHẦN B — Tìm lỗi trong code

Mỗi đoạn có **đúng một lỗi**. Chỉ ra lỗi, giải thích **tại sao sai**, và viết lại cho đúng.

Lỗi số 1

```
<button type="button" disabled="{{ soLy() === 1 }}"></button>
```

Lỗi số 2

```
// File: src/app/drink-detail/drink-detail.ts
import { Component, input } from '@angular/core';
import { DrinkModel } from './models';

@Component({
  selector: 'app-drink-detail',
  templateUrl: './drink-detail.html',
```

```
    styleUrls: ['./drink-detail.css'],
  })
  export class DrinkDetail {
    readonly drink = input.required<DrinkModel>();
  }
```

Lỗi số 3

```
// File: src/app/drink-list/drink-list.ts
import { Component } from '@angular/core';
import { DrinkDetail } from '../drink-detail/drink-detail';

@Component({
  selector: 'app-drink-list',
  imports: [],
  templateUrl: './drink-list.html',
  styleUrls: ['./drink-list.css'],
})
export class DrinkList {}
```

```
<!-- drink-list.html -->
<app-drink-detail [drink]="selectedDrink()" />
```

Lỗi số 4

```
<app-drink-detail [drink]="selectedDrink" />
```

Lỗi số 5

```
export class DrinkDetail {
  protected readonly drink = input.required<DrinkModel>();
}
```

Lỗi số 6

```
<ul>
  @for (topping of toppingCanDung()) {
    <li>{{ topping.name }}</li>
  }
</ul>
```

Lỗi số 7

```
import { Component, signal } from '@angular/core';
import { NgIf } from '@angular/common';

@Component({
  selector: 'app-drink-list',
  imports: [NgIf],
  templateUrl: './drink-list.html',
  styleUrls: ['./drink-list.css'],
})
export class DrinkList {
  protected readonly drinks = signal(MOCK_DRINKS);
}
```

```
@if (drink.isPopular) {
  <span>🔥 </span>
}
```

Lỗi số 8

```
@for (drink of drinks(); track drink.id) {
  <button [class.is-active]="selectedDrink().id === drink.id">{{ drink.name }}
</button>
}
```

Lỗi số 9

```
@for (drink of drinks; track drink.id) {
  <li>{{ drink.name }}</li>
}
```



PHẦN C — Project: Quầy Trà Sữa 🥤

Sáu nhiệm vụ, mỗi nhiệm vụ ôn một bài. Làm tuần tự vì nhiệm vụ sau dựa trên nhiệm vụ trước.

Nhiệm vụ C7 — ôn Bài 7: *Property Binding*

Chuẩn bị: thêm `imageUrl: string;` vào interface `DrinkModel`, rồi điền địa chỉ ảnh cho cả 3 loại trong `mock-drinks.ts`. Bạn tự tìm ảnh trên mạng, hoặc dùng tạm 3 địa chỉ này:

```
https://images.unsplash.com/photo-1558857563-b371033873b8?w=600
https://images.unsplash.com/photo-1515823064-d6e0c04616a7?w=600
https://images.unsplash.com/photo-1571934811356-5cc061b6821f?w=600
```

Yêu cầu: Màn hình hiển thị ảnh của loại trà sữa đang chọn. Địa chỉ ảnh phải lấy động từ signal, không được viết cứng. Ảnh cần mô tả thay thế cho người khiếm thị, và mô tả đó cũng đổi theo loại đang chọn. Giới hạn chiều rộng ảnh khoảng 300px.

Ngoài ra, nút — phải **tự khóa lại** khi số ly đang là 1.

Nhiệm vụ C8 — ôn Bài 8: Tạo & lồng Component

```
ng generate component drink-list
```

Yêu cầu: Chuyển **toàn bộ** logic và giao diện liên quan tới trà sữa sang **DrinkList**. Sau khi xong, class **App** chỉ còn giữ đúng signal tên quán, và **app.html** chỉ còn dòng tiêu đề cùng lời gọi component con.

⚠ Nhớ sửa đường dẫn **import** từ **./** thành **../**, và đừng làm rơi mất đoạn code nào.

Nhiệm vụ C9 — ôn Bài 9: **input()**

```
ng generate component drink-detail
```

Yêu cầu: Tách đôi trách nhiệm. **DrinkList** **chỉ lo việc chọn loại** — giữ 3 nút và signal ghi nhớ loại đang chọn. Mọi thứ còn lại chuyển sang **DrinkDetail**: tên, mô tả, giá, ảnh, bộ đếm số ly, tổng tiền, danh sách topping.

DrinkDetail **không được tự import** **MOCK_DRINKS**. Nó phải nhận dữ liệu qua một cổng **input bắt buộc**. Signal số ly và hai phương thức tăng/giảm trở thành state cục bộ của **DrinkDetail**. Hai **computed** cũng chuyển sang và lấy dữ liệu từ cổng input.

Nhiệm vụ C10 — ôn Bài 10: CSS & Flexbox

Yêu cầu: Viết CSS thuần vào **drink-detail.css** (và **drink-list.css** nếu muốn) để đạt **ba tiêu chí**:

- **Flexbox** — nội dung chính xếp dọc gọn gàng; cụm — / số ly / + nằm thẳng hàng ngang và căn giữa
- **Thứ bậc thị giác** — tên loại trà sữa to và đậm hơn hẳn dòng mô tả; giá và tổng tiền cũng nên nổi bật
- **Khoảng trắng** — dùng **padding**, **margin**, **gap**, **line-height** để nội dung có chỗ thở

Không có đáp án duy nhất. Cứ chọn màu và cỡ chữ theo gu của bạn.

Nhiệm vụ C11 — ôn Bài 11: **@for**

Yêu cầu: Thay khối `<pre>{{ ... | json }}</pre>` bằng một danh sách topping tử tế. Mỗi topping một dòng: **tên** bên trái, **số lượng kèm đơn vị** bên phải. Có khối dự phòng khi danh sách rỗng.

Dọn sạch dấu vết: thẻ `<pre>` và `JsonPipe` (cả dòng `import` lẫn mảng `imports`).

💡 **Topping** không có trường `id`. Tự chọn trường phù hợp làm `track` và nghĩ xem vì sao.

Nhiệm vụ C12 — ôn Bài 12: `@if`

Chuẩn bị: thêm `isPopular: boolean;` vào `DrinkModel`, rồi điền giá trị cho 3 loại — cho **2 loại** là `true` và **1 loại** là `false`.

Yêu cầu: Hai phần.

Phần 1 — làm cho cái tên `DrinkList` thành thật. Component phải nắm giữ **cả danh sách** trà sữa và **tự sinh nút cho từng loại** bằng vòng lặp, thay vì 3 nút viết tay. Ba phương thức `chonTranChau` / `chonMatcha` / `chonHongTra` gộp lại thành **một** phương thức nhận tham số.

Phần 2 — tính năng mới. Loại nào được đánh dấu phổ biến thì nút của nó hiện thêm huy hiệu 🔥. Huy hiệu này **mang thông tin thật** nên nhớ xử lý đúng cho trình đọc màn hình (khác với emoji trang trí).

🎯 **Phép thử:** thêm loại trà sữa thứ tư vào `mock-drinks.ts`. Nếu nút của nó **tự xuất hiện** mà bạn không sửa dòng HTML nào, bạn đã làm đúng tinh thần của bài.

🚀 PHẦN D — Thử thách nâng cao (bắt buộc)

- Huy hiệu "đắt nhất".** Dùng `computed` tìm loại có `giaCoBan` cao nhất, rồi dùng `@if` hiện nhãn 💎 trên nút của nó. Gợi ý: `Math.max(...)` kết hợp `.map()`.
- Ba nhánh giá.** Dùng `@if` / `@else if` / `@else` hiện nhãn giá theo mức: dưới 40000đ là "Bình dân", từ 40000 đến 50000đ là "Tiêu chuẩn", trên 50000đ là "Cao cấp".
- Đánh số topping.** Dùng biến `$index` để hiện số thứ tự trước tên mỗi topping. Nhớ rằng `$index` đếm từ 0.
- Tô màu dòng chẵn lẻ.** Dùng `$even` cùng `[class.row-alt]` để các dòng topping so le màu nền cho dễ đọc.
- Giảm giá theo số lượng.** Thêm `computed` tính tổng tiền sau giảm: từ 5 ly trở lên giảm 10%. Dùng `@if` chỉ hiện dòng "Đã giảm 10%" khi điều kiện thỏa.

📋 Bảng tự theo dõi tiến độ

Phần	Nội dung	Đã xong?
A	12 câu lý thuyết	☐
B	9 đoạn code tìm lỗi	☐

Phần	Nội dung	Đã xong?
C7	<code>[src] [alt] [disabled]</code>	☐
C8	Tạo <code>DrinkList</code> , ba bước lồng component	☐
C9	Tạo <code>DrinkDetail</code> với <code>input.required()</code>	☐
C10	CSS thuần: Flexbox + thứ bậc + khoảng trắng	☐
C11	<code>@for</code> + <code>track</code> + <code>@empty</code>	☐
C12	<code>@if</code> + vòng lặp nút + gộp phương thức	☐
D1	Huy hiệu đắt nhất	☐
D2	Ba nhánh giá với <code>@else if</code>	☐
D3	Đánh số topping bằng <code>\$index</code>	☐
D4	Tô màu so le bằng <code>\$even</code>	☐
D5	Giảm giá từ 5 ly	☐

Sổ tay tra cứu nhanh (Bài 7 → 12)

Bốn cách gắn dữ liệu vào template

Ký hiệu	Tên	Chiều	Ví dụ
<code>{{ }}</code>	Interpolation	Class  Màn hình	<code>{{ drink().name }}</code>
<code>[]</code>	Property binding	Class  Màn hình	<code>[src]="drink().imageUrl"</code>
<code>[attr.____]</code>	Attribute binding	Class  Màn hình	<code>[attr.aria-label]="Xóa"</code>
<code>()</code>	Event binding	Màn hình  Class	<code>(click)="chon(drink)"</code>

Quy tắc: có `[ ]` thì bên trong là **code TypeScript**; không có `[ ]` thì bên trong là **chữ thuần** (chỉ phần `{{ }}` được thay giá trị). Đừng trộn hai kiểu:

```
aria-label="Xóa + {{ name }}"
<!-- ❌ dấu + bị đọc thành chữ "cộng" -->
aria-label="Xóa {{ name }}"
<!-- ✅ interpolation -->
[attr.aria-label]="Xóa ' + name"
<!-- ✅ binding, + nối chuỗi thật -->
```

Ba bước lồng component

```
// 1. import class con
import { DrinkDetail } from '../drink-detail/drink-detail';

@Component({
  imports: [DrinkDetail], // 2. thêm vào mảng imports
  // ...
})
```

```
<app-drink-detail />
<!-- 3. dùng selector -->
```

Đường dẫn tương đối

```
src/app/
├── models.ts           ← đích đến
├── app.ts              → './models'
├── drink-list/
│   └── drink-list.ts   → '../models' (đi lùi ra 1 cấp)
```

input() — nhận dữ liệu từ cha

```
// Con: KHÔNG có protected, CHỈ có readonly
readonly drink = input.required<DrinkModel>(); // bắt buộc
readonly size = input<string>('M');           // có mặc định
```

```
<!-- Cha: truyền GIÁ TRỊ (có dấu ngoặc), không truyền signal -->
<app-drink-detail [drink]="selectedDrink()" />
```

Ba loại dữ liệu trong một component con

Loại	Ai sở hữu	Ví dụ
input()	Cha đưa xuống	readonly drink = input.required<...>()
signal()	Con tự quản lý	protected readonly soLy = signal(1)
computed()	Tính từ cả hai	computed(() => this.drink().gia * this.soLy())

@for — lặp danh sách


```
<ul>
  @for (drink of drinks(); track drink.id) {
    <li>{{ $index + 1 }}. {{ drink.name }}</li>
  } @empty {
    <li>Chưa có loại nào.</li>
  }
</ul>
```

Biến có sẵn: `$index` (đếm từ 0), `$first`, `$last`, `$even`, `$odd`, `$count`.

Chọn `track`: một giá trị duy nhất và không đổi. `id` là tốt nhất, `name` tạm ổn, `$index` chỉ dùng khi đường cùng.

@if — hiển thị có điều kiện

```
@if (drink.isPopular) {
  <span>🔥 </span>
} @else if (drink.gia > 50000) {
  <span>💎 </span>
} @else {
  <span>—</span>
} @if (currentUser(); as user) {
  <p>{{ user.name }}</p>
}
```

`@if` xóa hẳn thẻ khỏi DOM, khác `display: none` chỉ ẩn đi.

📌 `@for`, `@if`, `@switch`, `@empty` là cú pháp tích hợp sẵn — KHÔNG import gì cả. `NgFor` / `NgIf` / `CommonModule` là đồ của thế hệ cũ (`*ngFor`, `*ngIf`), đã bị khai tử ở v20.

Truyền tham số vào hàm xử lý sự kiện

```
<button (click)="chonMon(drink)">{{ drink.name }}</button>
```

```
protected chonMon(drink: DrinkModel): void {
  this.selectedDrink.set(drink);
}
```

Một hàm dùng cho mọi món — thêm 50 món cũng không phải viết thêm hàm.

CSS trong Angular

```

/* drink-detail.css – chỉ ảnh hưởng template của DrinkDetail, không rò rỉ
ra ngoài */

:host {
  /* nhắm vào chính thẻ <app-drink-detail> */
  display: block;
}

.card {
  display: flex;
  flex-direction: column; /* xếp dọc */
  gap: 1rem; /* khoảng cách giữa các con */
  padding: 1.5rem; /* đệm bên trong */
}

.qty-row {
  display: flex;
  align-items: center; /* căn giữa theo trục dọc */
  justify-content: space-between; /* đẩy hai đầu ra xa */
  gap: 0.75rem;
}

```

Định dạng có điều kiện — dùng `[class.____]`, **không** dùng `ngClass`:

```
<button [class.is-active]="selectedDrink().id === drink.id">...</button>
```

Khả năng tiếp cận — nhắc nhanh

Tình huống	Cách làm
Ảnh mang nội dung	<code>[alt]=" 'Ảnh ' + drink().name "</code>
Nút chỉ có ký hiệu (+, -)	<code>aria-label="Tăng số ly"</code>
Emoji trang trí	<code>aria-hidden="true"</code>
Emoji mang thông tin	<code>aria-label="Món phổ biến"</code>
Nút bật/tắt trạng thái chọn	<code>[attr.aria-pressed]="dangChon"</code>
Giá trị thay đổi cần thông báo	<code>aria-live="polite"</code>
Thẻ tiêu đề	Đi tuần tự <code>h1</code> → <code>h2</code> → <code>h3</code> , không nhảy cóc

Những cái bẫy hay dính

Bẫy	Sai	Đúng
Gán vào thuộc tính bằng <code>{{ }}</code>	<code>disabled="{{ x() }}"</code>	<code>[disabled]="x()"</code>

Bẫy	Sai	Đúng
Truyền signal thay vì giá trị	<code>[drink]="selected"</code>	<code>[drink]="selected()"</code>
Đặt <code>protected</code> cho input	<code>protected readonly d = input...</code>	<code>readonly d = input...</code>
Quên thêm vào mảng <code>imports</code>	<code>imports: []</code>	<code>imports: [DrinkDetail]</code>
Sai đường dẫn sau khi chuyển file	<code>'./models'</code>	<code>'../models'</code>
Thiếu <code>track</code>	<code>@for (d of ds())</code>	<code>@for (d of ds()); track d.id)</code>
Quên <code>()</code> sau signal trong <code>@for</code>	<code>@for (d of ds; ...)</code>	<code>@for (d of ds(); ...)</code>
Import <code>NgFor</code> / <code>NgIf</code>	<code>imports: [NgFor]</code>	Không import gì cả
Một dấu <code>=</code> trong điều kiện	<code>[class.a]="x().id = y.id"</code>	<code>[class.a]="x().id === y.id"</code>